

FIAP - Engenharia de Software 2.0

Trabalho: Auditoria e Refatoração com IA

Integrantes do Grupo

RM368446 - Caio Silva

RM368580 - Fábio Luiz

RM368316 - Higor Robles

RM368253 - João Lucas

RM367683 - Vitor Pantoja

GIT REPO

<https://github.com/caio-swdev/fiap-pixel-press>

PROMPTS

Inicial

Aja como um Arquiteto de Software, como se você viesse de fora auditar o código. Analise meu projeto e gere um relatório de dívida técnica, focando em violações de SOLID, kiss, yagni, clean architecture, e acoplamento excessivo. esse é caminho da base de código:

/home/caio/github/fiap-pixel-press/src

Refatoração

Refatore com base no relatório:

[relatorio-divida-tecnica.md](#)

Relatório de Dívida Técnica — fiap-pixel-press/src

Auditoria de arquitetura conduzida na perspectiva de um arquiteto externo.
Foco: violações de SOLID, KISS, YAGNI, Clean Architecture e acoplamento excessivo.

Escopo: NestJS API (`api/src`) + React SPA (`web/src`), ~5.4k LOC reais, 97 arquivos.

Veredito geral: base **acima da média**. Camadas limpas, DIP bem aplicado no ponto certo, envelope de erro consistente, segurança de credenciais cuidadosa. A dívida é **modesta e localizada**, não estrutural. Abaixo, do mais grave ao cosmético.

Pontos fortes (para calibrar a régua)

#	Acerto	Onde
1	DIP/OCP textbook: <code>RawgClient</code> atrás de token + factory por env	<code>jogos.module.ts:22</code> , <code>rawg.types.ts:38</code>
2	Envelope de erro único, domínio desacoplado do transporte HTTP	<code>all-exceptions.filter.ts</code> , <code>domain.exception.ts</code>
3	Fluxo web em camadas estritas: <code>http</code> → <code>endpoints</code> → <code>hooks</code> → <code>components</code>	<code>endpoints.ts:3</code> ("componentes nunca chamam isto direto")
4	Segredos nunca vazam: redact de logs, key só em query, refresh single-flight	<code>app.module.ts:26</code> , <code>http-rawg.client.ts:111</code> , <code>http.ts:78</code>

Dívida — alta severidade

#1 — Anemic Domain Model + ausência de camada de repositório (Clean Architecture)

Todos os services injetam `PrismaService` direto e operam sobre registros do ORM. Não há entidades de domínio com comportamento, nem porta de persistência. A regra de negócio (`validarNota` , `ownership` , `duplicidade`) vive espalhada em services acoplados ao Prisma.

- **Impacto:** regra de domínio não é testável sem banco; trocar ORM toca todos os services; a "dependency rule" do Clean Arch (domínio não conhece infra) está invertida.
- **Trade-off honesto:** para um MVP CRUD isto é **defensável** (repository pattern sobre Prisma frequentemente é cerimônia inútil). Vira dívida real **só** quando a lógica por entidade crescer. Não refatore ainda; registre a decisão.

#2 — Acoplamento cruzado entre features: `ReviewsController` importa `ModeracaoService`

`reviews.controller.ts:32` injeta `ModeracaoService` só para servir `PATCH /reviews/:id/hide` . O endpoint foi colocado por conveniência de URL, não por ownership do domínio. Moderação vaza para dentro de reviews.

- **Violação:** SRP no controller + acoplamento entre módulos que deveriam ser independentes.
- **Correção:** mover `hide` para o `ModeracaoController` (ex.: `POST /moderation/reviews/:id/hide`) ou expor via evento. Baixo custo, alto ganho de fronteira.

#3 — `JogosService.garantirReferencia` mistura duas responsabilidades (SRP)

`jogos.service.ts:71` é ao mesmo tempo leitor cache-aside da RAWG e escritor no banco (`upsert`). `ReviewsService` e `BibliotecaService` chamam o catálogo para obter um **efeito colateral de persistência**, o que é surpreendente: um service "de catálogo read-only" grava no DB.

- **Correção:** extrair `ReferenciaJogoService` (ou método em repositório de Jogo) responsável por materializar a referência. Catálogo lê; outro colaborador persiste.

#8 — CacheService : Map in-process sem limite de tamanho (vetor de OOM / DoS)

cache.service.ts:16 só evicta por TTL **na leitura** da própria chave. Entrada nunca relida **nunca é apagada**, mesmo expirada (não há sweep em background). O Map só cresce.

- **Por que é alta, não cosmético:**

- i. A chave é rawg:busca:\${search}:... com search string livre → espaço de chaves **infinito**.
 - ii. /games é **público** (jogos.controller.ts , sem guard) → gatilho externo **sem autenticação**.
 - iii. Um loop de ?search=<aleatório> enche o Map até o processo morrer = **DoS trivial por memória**.
 - iv. Render free = 512MB; com entradas de ~5-15KB (detalhe carrega screenshots[]), ~40-80k chaves já estouram, alcançável em **minutos**.
- **Blast radius:** disponibilidade (processo cai e reinicia), **não** integridade de dados nem vazamento de segredo. O cache só guarda resposta pública da RAWG.
 - **Trade-off:** o comentário do arquivo justifica *não usar Redis no MVP*; **não** justifica Map sem teto. São coisas diferentes.
 - **Correção (trivial, ~10 linhas, sem dependência nova):** cap de tamanho + descarte do mais antigo.

```
private readonly maxEntradas = 1000;
```

```
async set<T>(chave: string, valor: T): Promise<void> {  
  if (this.store.size >= this.maxEntradas) {  
    // Map mantém ordem de inserção: a primeira chave é a mais antiga.  
    const maisAntiga = this.store.keys().next().value;  
    if (maisAntiga !== undefined) this.store.delete(maisAntiga);  
  }  
  this.store.set(chave, { valor, expiraEm: Date.now() + this.ttlMs });  
}
```

O cap transforma "ilimitado" em "teto fixo de N entradas", que é o que mata o vetor de OOM. LRU de verdade (renovar posição no acesso) é refinamento opcional.

Dívida — média severidade

#4 — God component: `GameDetailPage.tsx` (497 LOC)

Página + 5 modais (`AddBiblioteca` , `ReviewForm` , `Report` , `Hide` , `excluir inline`) + 4 schemas zod num arquivo só. SRP e KISS.

- **Correção:** extrair cada modal para `pages/game-detail/` ou `components/` . Cada um já é função isolada, é recorte mecânico.

#5 — YAGNI explícito: 3 módulos vazios

`ListasModule` , `SocialModule` , `WishlistModule` são `@Module({})` vazios, importados no `app.module.ts:38` "para preservar a estrutura". Estrutura especulativa para features não construídas é o caso-livro de YAGNI.

- **Trade-off:** como scaffolding de assignment é justificável e o custo é ~zero. Mas não os importe no `AppModule` até existirem; arquivo vazio no disco já preserva a intenção sem poluir o grafo de DI.

#6 — Duplicação de enums entre stacks (DRY + risco de drift)

- Hierarquia de papéis definida **duas vezes**: `NIVEL_PAPEL` (API) e `NIVEL` em `session.tsx:78` (web). Divergem silenciosamente se um lado mudar.
- Lista de status de biblioteca duplicada **dentro do mesmo arquivo** em `GameDetailPage.tsx` : `STATUS_OPCOES` (linha 20) e `z.enum([...])` (linha 262), **em ordens diferentes**. Adicionar um status exige editar dois lugares; esquecer um quebra o form ou a validação.
- **Correção:** derivar o `z.enum` de `STATUS_OPCOES` (`z.enum(STATUS_OPCOES as [...])`). Cross-stack não há fix limpo sem pacote compartilhado; registre como aceito.

#7 — `carregarComOwnership` duplicado verbatim

Padrão idêntico (`findUnique` → 404 → `check dono` → 403) em `reviews.service.ts:112` e `biblioteca.service.ts:90` . DRY.

- **Correção:** helper genérico `assertOwnership(entity, usuarioId)` no `common/` . Baixa prioridade.

Dívida — baixa / cosmético

#	Item	Local	Nota
9	Check-then-create em review/biblioteca é race; sob concorrência o findUnique perde e o unique constraint dispara P2002 → 500 em vez de 409	reviews.service.ts:35 , biblioteca.service.ts:32	Capturar P2002 e remapear para ConflitoException
10	basePath hardcoded ('/api/v1/reviews' , '/api/v1/games') repetido nos controllers; duplica o global prefix	reviews.controller.ts:38 , jogos.controller.ts:14	Centralizar prefixo numa const; drift se a versão mudar
11	Validação de domínio dividida: validarNota no service + faixa também no DTO/web	reviews.service.ts:103	Aceitável (defesa em profundidade), mas a fonte da verdade fica ambígua
12	DomainException.details quase sempre [] ; campo carregado mas subutilizado	domain.exception.ts:13	YAGNI leve no contrato de erro

Recomendação priorizada

1. **Faça agora (alto valor, baixo custo):** #8 (cap no cache — vetor de OOM), #2 (mover hide p/ moderação), #6 (derivar z.enum), #9 (mapear P2002).
2. **Faça quando tocar o arquivo:** #4 (quebrar GameDetailPage), #7 (helper de ownership).
3. **Apenas registre como ADR, não refatore:** #1 (sem repositório), #3 (materialização de referência), #5 (módulos vazios). São trade-offs conscientes de MVP, não erros.

Resumo de uma linha: não é uma base com dívida técnica, é uma base limpa com ~12 arestas, das quais 4 valem corrigir esta semana (com destaque para o #8, que é availability) e o resto é nota-de-rodapé de MVP.